



CELEBAL
TECHNOLOGIES

SHOP EASE E-COMMERCE DATABASE ANALYSIS

Celebal Summer Internship 2026

Week 2 Assignment

Task 2

Dataset: ShopEaseDB

Submission Date:

01-05-2026

Prepared By:

Keshav Kumar sharma

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

SQL Query:

```
SELECT *
```

```
FROM customers;
```

Explanation:

This query displays all columns and all records present in the customers table.

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Q2. Retrieve only the first_name, last_name, and city of all customers.

SQL Query:

```
SELECT first_name, last_name, city
```

```
FROM customers;
```

Explanation:

This query returns only the first name, last name, and city of every customer.

	first_name	last_name	city
1	Aarav	Sharma	Mumbai
2	Priya	Patel	Ahmedabad
3	Rohan	Gupta	Delhi
4	Sneha	Reddy	Hyderabad
5	Vikram	Singh	Jaipur
6	Ananya	Iyer	Chennai
7	Karan	Mehta	Pune
8	Divya	Nair	Kochi

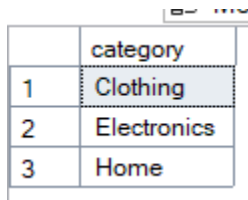
Q3. List all unique categories available in the products table.

SQL Query:

```
SELECT DISTINCT category  
FROM products;
```

Explanation:

The DISTINCT keyword removes duplicate values and returns only unique product categories.



	category
1	Clothing
2	Electronics
3	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Primary Keys:

customers → customer_id

products → product_id

orders → order_id

order_items → item_id

Explanation:

A Primary Key uniquely identifies each record in a table.

- It must be UNIQUE so that no two rows can have the same identifier.
- It cannot be NULL because every row must have a valid identifier.
- Primary Keys help maintain data integrity and establish relationships between tables

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Constraints on email column:

1. UNIQUE
2. NOT NULL

Example:

```
INSERT INTO customers
VALUES (
109,
'Test',
'User',
'aarav.s@email.com',
'Delhi',
'Delhi',
'2024-09-01',
1
);
```

Explanation:

The email 'aarav.s@email.com' already exists in the table.

SQL Server will generate an error because the UNIQUE constraint prevents duplicate email addresses.

Expected Error:

Msg 2627, Level 14, State 1, Line 134 Violation of UNIQUE KEY constraint 'UQ__customer__AB6E616481D903DC'. Cannot insert duplicate key in object 'dbo.customers'. The duplicate key value is (aarav.s@email.com).

Msg 2627, Level 14, State 1, Line 134
Violation of UNIQUE KEY constraint 'UQ__customer__AB6E616481D903DC'. Cannot insert duplicate key in object 'dbo.customers'. The duplicate key value is (aarav.s@email.com).
The statement has been terminated.

Completion time: 2026-06-01T12:09:34.1602761+05:30

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it?

Q6. Try inserting a product with unit_price = -50. What happens and which constraint prevents it?

SQL Query:

```
INSERT INTO products
VALUES (
209,
'Test Product',
'Electronics',
'TestBrand',
-50,
100
);
```

Explanation:

The insertion fails because the products table contains a CHECK constraint:

CHECK (unit_price > 0)

Since -50 is less than 0, SQL Server rejects the row and displays a CHECK constraint violation error.

Expected Error:

The INSERT statement conflicted with the CHECK constraint.

Msg 547, Level 16, State 0, Line 134
The INSERT statement conflicted with the CHECK constraint "CK_products_unit_p_4D94879B". The conflict occurred in database "ShopEaseDB", table "dbo.products", column 'unit_pr
The statement has been terminated.

Completion time: 2026-06-01T12:08:35.4397376+05:30

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

SQL Query:

```
SELECT *  
FROM orders  
WHERE status = 'Delivered';
```

Explanation:

This query returns all orders that have been successfully delivered to customers.

	order_id	customer_id	order_date	status	total_amount
1	1001	101	2024-08-01	Delivered	4498.00
2	1002	102	2024-08-03	Delivered	799.00
3	1004	101	2024-08-10	Delivered	3499.00
4	1006	105	2024-08-15	Delivered	5898.00
5	1008	103	2024-08-20	Delivered	899.00
6	1010	108	2024-08-28	Delivered	1598.00

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

SQL Query:

```
SELECT *  
FROM products  
WHERE category = 'Electronics'  
AND unit_price > 2000;
```

Explanation:

This query filters products belonging to the Electronics category and displays only those with a price greater than ₹2000.

	product_id	product_name	category	brand	unit_price	stock_qty
1	203	Smart Watch	Electronics	Noise	2999.00	150
2	205	Bluetooth Speaker	Electronics	JBL	3499.00	200

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

SQL Query:

```
select * from customers
```

```
where join_date between '2024-01-01' and '2024-12-31'
```

```
and state = 'Maharashtra'
```

Explanation:

This query retrieves customers from Maharashtra who joined during the year 2024.

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

SQL Query:

```
SELECT *
```

```
FROM orders
```

```
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
```

```
AND status <> 'Cancelled';
```

Explanation:

This query returns orders placed between August 10 and August 25, 2024, excluding cancelled orders.

	order_id	customer_id	order_date	status	total_amount
1	1004	101	2024-08-10	Delivered	3499.00
2	1006	105	2024-08-15	Delivered	5898.00
3	1007	106	2024-08-18	Pending	1299.00
4	1008	103	2024-08-20	Delivered	899.00
5	1009	107	2024-08-25	Shipped	6098.00

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

Answer:

The idx_orders_date index is created on the order_date column of the orders table.

Purpose:

- It speeds up searches based on order dates.
- SQL Server can quickly locate matching rows instead of scanning the entire table.
- It improves performance for filtering, sorting, and range-based queries involving order_date.

Example Query:

```
SELECT *  
FROM orders  
WHERE order_date BETWEEN '2024-08-01' AND '2024-08-31';
```

Explanation:

The database engine can use the index to directly find rows within the specified date range, reducing query execution time.

	order_id	customer_id	order_date	status	total_amount
1	1001	101	2024-08-01	Delivered	4498.00
2	1002	102	2024-08-03	Delivered	799.00
3	1003	103	2024-08-05	Shipped	7498.00
4	1004	101	2024-08-10	Delivered	3499.00
5	1005	104	2024-08-12	Cancelled	2999.00
6	1006	105	2024-08-15	Delivered	5898.00
7	1007	106	2024-08-18	Pending	1299.00
8	1008	103	2024-08-20	Delivered	899.00
9	1009	107	2024-08-25	Shipped	6098.00
10	1010	108	2024-08-28	Delivered	1598.00

Q12. If you run: SELECT * FROM customers WHERE YEAR(join_date) = 2024; — would the index on join_date be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Answer:

No, the index would generally not be used efficiently.

Reason:

Applying the YEAR() function to join_date forces SQL Server to evaluate the function for every row before filtering. This prevents efficient index usage and may result in a table scan.

Non-SARGable Query:

```
SELECT *  
FROM customers  
WHERE YEAR(join_date) = 2024;
```

SARGable Query:

```
SELECT *  
FROM customers  
WHERE join_date >= '2024-01-01'  
AND join_date < '2025-01-01';
```

Explanation:

The rewritten query allows SQL Server to use an index on join_date efficiently because it compares the column directly without applying a function.

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
1	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
2	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
3	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
4	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
5	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
6	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
7	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
8	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

SQL Query:

```
SELECT COUNT(*) AS Total_Orders  
FROM orders;
```

Explanation:

This query counts the total number of records present in the orders table.

	total_orders
1	10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

SQL Query:

```
SELECT SUM(total_amount) AS total_revenue  
FROM orders  
WHERE status = 'Delivered';
```

Explanation:

This query calculates the total revenue generated from orders that have been delivered successfully.

	total_revenue
1	17191.00

Q15. Calculate the average unit_price of products in each category.

SQL Query:

```
SELECT  
category,  
AVG(unit_price) AS Average_Price  
FROM products  
GROUP BY category;
```

Explanation:

This query groups products by category and calculates the average product price for each category.

	category	Average_price
1	Clothing	2699.000000
2	Electronics	2224.000000
3	Home	949.000000

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

SQL Query:

```
SELECT
status,
COUNT(order_id) AS Order_Count,
SUM(total_amount) AS Total_Revenue
FROM orders
GROUP BY status
ORDER BY Total_Revenue DESC;
```

Explanation:

This query displays the number of orders and total revenue for each order status and sorts the results by revenue from highest to lowest.

	status	Order_Count	Total_Revenue
1	Delivered	6	17191.00
2	Shipped	2	13596.00
3	Cancelled	1	2999.00
4	Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

SQL Query:

```
--most expensive product name SELECT category, product_name, unit_price FROM
products p WHERE unit_price = ( SELECT MAX(unit_price) FROM products WHERE category
= p.category );
```

```
--cheapest product name SELECT Category, product_name, unit_price FROM products p
WHERE unit_price = ( SELECT MIN(unit_price) FROM products WHERE category =
p.category );
```

Explanation:

This query finds the highest and lowest product prices within each category.

	category	product_name	unit_price
1	Home	Bedsheet Set	1299.00
2	Electronics	Bluetooth Speaker	3499.00
3	Clothing	Running Shoes	4599.00

	Category	product_name	unit_price
1	Clothing	Cotton T-Shirt	799.00
2	Electronics	Laptop Stand	899.00
3	Home	Cushion Covers (Set)	599.00

**Q18. List all product categories where the average unit_price is greater than ₹2000.
(Hint: Use HAVING clause)**

SQL Query:

```
SELECT
category,
AVG(unit_price) AS Average_Price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
```

Explanation:

The HAVING clause filters grouped results and returns only those categories whose average product price exceeds ₹2000

	category	Average_Price
1	Clothing	2699.000000
2	Electronics	2224.000000

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

```
SELECT
o.order_id,
o.order_date,
c.first_name,
c.last_name,
o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;
```

Explanation:

The INNER JOIN returns only those records where matching customer_id values exist in both tables.

	order_id	order_date	first_name	last_name	total_amount
1	1001	2024-08-01	Aarav	Sharma	4498.00
2	1002	2024-08-03	Priya	Patel	799.00
3	1003	2024-08-05	Rohan	Gupta	7498.00
4	1004	2024-08-10	Aarav	Sharma	3499.00
5	1005	2024-08-12	Sneha	Reddy	2999.00
6	1006	2024-08-15	Vikram	Singh	5898.00
7	1007	2024-08-18	Ananya	Iyer	1299.00
8	1008	2024-08-20	Rohan	Gupta	899.00
9	1009	2024-08-25	Karan	Mehta	6098.00
10	1010	2024-08-28	Diya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

SQL Query:

```
SELECT  
  
c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.total_amount  
  
FROM customers c  
  
LEFT JOIN orders o  
  
ON c.customer_id = o.customer_id;
```

Explanation:

LEFT JOIN returns all rows from the customers table and matching rows from the orders table. If a customer has no orders, the order columns will contain NULL values.

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

SQL Query:

```
select oi.order_id , p.product_name , oi.quantity , oi.unit_price , oi.discount_pct  
  
FROM orders o  
  
INNER JOIN order_items oi  
  
ON o.order_id = oi.order_id  
  
INNER JOIN products p  
  
ON oi.product_id = p.product_id;
```

Explanation:

This query joins three tables to display detailed information about every product included in each order.

	order_id	product_name	quantity	unit_price	discount_pct
1	1001	Wireless Earbuds	2	1499.00	0.00
2	1001	Laptop Stand	1	899.00	10.00
3	1002	Cotton T-Shirt	1	799.00	0.00
4	1003	Smart Watch	1	2999.00	0.00
5	1003	Running Shoes	1	4599.00	5.00
6	1004	Bluetooth Speaker	1	3499.00	0.00
7	1005	Smart Watch	1	2999.00	0.00
8	1006	Wireless Earbuds	1	1499.00	10.00
9	1006	Running Shoes	1	4599.00	5.00
10	1007	Bedsheet Set	1	1299.00	0.00
11	1008	Laptop Stand	1	899.00	0.00
12	1009	Bluetooth Speaker	1	3499.00	0.00
13	1009	Cushion Covers (Set)	2	599.00	15.00
14	1010	Bedsheet Set	1	1299.00	0.00
15	1010	Cushion Covers (Set)	1	599.00	0.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

Answer:

LEFT JOIN:

Returns all rows from the left table and matching rows from the right table.

Example:

```
SELECT *
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;
```

This shows all customers, even those who have never placed an order.

RIGHT JOIN:

Returns all rows from the right table and matching rows from the left table.

Example:

```
SELECT *
FROM customers c
```

```
RIGHT JOIN orders o
ON c.customer_id = o.customer_id;
```

This shows all orders, even if a matching customer record does not exist.

FULL OUTER JOIN:

Returns all rows from both tables. Matching rows are combined, while non-matching rows appear with NULL values.

Use Case:

A FULL OUTER JOIN is useful when we want to identify records that exist in one table but not in the other.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Foreign Key Relationships:

1. orders.customer_id
→ references customers.customer_id
2. order_items.order_id
→ references orders.order_id
3. order_items.product_id
→ references products.product_id

Example:

```
INSERT INTO orders
VALUES
(
1011,
999,
GETDATE(),
'Pending',
1000
);
```

Explanation:

Since customer_id = 999 does not exist in the customers table, SQL Server will reject the insertion.

Expected Error:

The INSERT statement conflicted with the FOREIGN KEY constraint.

Reason:

Foreign Keys enforce referential integrity and ensure that related records exist in the parent table before insertion.

```
MSG 5774, Level 16, State 11, Line 4/3
The INSERT statement conflicted with the FOREIGN KEY constraint "FK_orders_customer_5535A963". The conflict occurred in database "ShopEaseDB", table "dbo.customers", column 'customer_id'.
The statement has been terminated.

Completion time: 2026-06-01T16:58:34.0364457+05:30
```

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers: • 'Budget' → unit_price < 1000 • 'Mid-Range' → unit_price BETWEEN 1000 AND 3000 • 'Premium' → unit_price > 3000
Display: product_name, unit_price, price_tier.

SQL Query:

SELECT

product_name, unit_price,

CASE

WHEN unit_price < 1000 THEN 'Budget'

WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'

ELSE 'Premium'

END AS price_tier FROM products;

Explanation:

The CASE statement works like an IF-ELSE condition in SQL.

Classification Rules:

- Budget → unit_price < 1000
- Mid-Range → unit_price BETWEEN 1000 AND 3000
- Premium → unit_price > 3000

The query displays each product along with its price and the corresponding price tier.

	product_name	unit_price	price_tier
1	Wireless Earbuds	1499.00	Mid-Range
2	Cotton T-Shirt	799.00	Budget
3	Smart Watch	2999.00	Mid-Range
4	Running Shoes	4599.00	Premium
5	Bluetooth Speaker	3499.00	Premium
6	Bedsheet Set	1299.00	Mid-Range
7	Laptop Stand	899.00	Budget
8	Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

Query :

```
select SUM(
```

```
  CASE
```

```
    WHEN status = 'Delivered' THEN 1
```

```
    ELSE 0
```

```
  END
```

```
) AS Delivered_orders ,
```

```
SUM(
```

```
  CASE
```

```
    WHEN status <> 'Delivered' THEN 1
```

```
    ELSE 0
```

END

) AS NOT_Delivered_orders

from orders;

	Delivered_orders	NOT_Delivered_orders
1	6	4

Q26. Explain each letter of ACID: • A – Atomicity • C – Consistency • I – Isolation • D – Durability Give a real-world example (e.g., bank transfer) showing why each property is important.

ACID is a set of properties that ensure database transactions are processed reliably and accurately.

A – Atomicity

Definition:

Atomicity means a transaction is treated as a single unit of work. Either all operations are completed successfully, or none of them are performed.

Bank Transfer Example:

Suppose ₹1000 is transferred from Account A to Account B.

Step 1: Deduct ₹1000 from Account A.

Step 2: Add ₹1000 to Account B.

If the system crashes after Step 1 but before Step 2, Atomicity ensures that the deduction is rolled back so that no money is lost.

Importance:

Prevents partial transactions and keeps data accurate.

C – Consistency

Definition:

Consistency ensures that a transaction takes the database from one valid state to another valid state while following all rules, constraints, and relationships.

Bank Transfer Example:

Before the transfer, the total amount of money in the system is valid.

After the transfer, the total amount should remain the same.

No database constraints or business rules should be violated.

Importance:

Maintains data integrity and correctness.

I – Isolation

Definition:

Isolation ensures that multiple transactions running at the same time do not interfere with each other.

Bank Transfer Example:

Two users try to withdraw money from the same account simultaneously.

Isolation ensures that each transaction is processed independently, preventing incorrect balances.

Importance:

Prevents conflicts and inconsistent results during concurrent operations.

D – Durability

Definition:

Durability guarantees that once a transaction is committed, its changes are permanently stored in the database.

Bank Transfer Example:

After a successful transfer is committed, the updated balances remain saved even if the system crashes or loses power immediately afterward.

Importance:

Ensures that committed data is never lost.

Q27. Write a SQL transaction that does the following atomically: 1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00) 2. Insert two order items for that order 3. Update the stock_qty of the purchased products 4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT. Write the complete BEGIN...COMMIT/ROLLBACK block.

SQL Query:

BEGIN TRY

BEGIN TRANSACTION;

-- Step 1: Insert New Order

INSERT INTO orders

(
 order_id,
 customer_id,
 order_date,
 status,
 total_amount

)

VALUES

(
 1011,
 102,

```

        GETDATE(),
        'Pending',
        1598.00
    );

-- Step 2: Insert Order Items
INSERT INTO order_items
(
    item_id,
    order_id,
    product_id,
    quantity,
    unit_price,
    discount_pct
)
VALUES
(5016,1011,206,1,1299.00,0);

INSERT INTO order_items
(
    item_id,
    order_id,
    product_id,
    quantity,
    unit_price,
    discount_pct
)
VALUES
(5017,1011,208,1,599.00,0);

-- Step 3: Update Product Stock
UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 206;

UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 208;

```

```
-- Step 4: Commit Transaction
COMMIT TRANSACTION;

PRINT 'Transaction Completed Successfully';

END TRY

BEGIN CATCH

ROLLBACK TRANSACTION;

PRINT 'Transaction Failed - Changes Rolled Back';

END CATCH;
```

Explanation:

1. A new order is inserted into the orders table.
2. Two order items are inserted into the order_items table.
3. Stock quantities of the purchased products are reduced.
4. If every statement executes successfully, COMMIT saves all changes permanently.
5. If any statement fails (such as a foreign key violation, duplicate primary key, or constraint violation), the CATCH block executes and ROLLBACK removes all changes made during the transaction.

This ensures Atomicity (the "A" in ACID): either all operations succeed or none of them are applied.